

по результатам нашего общения предлагаю вот такую таблицу размещения пинов:

Итоговая таблица пинов (окончательная версия)

| Пин | Назначение | Тип | Примечание |
|-----|------------|-----|------------|
|-----|------------|-----|------------|

|        |        |        |                  |
|--------|--------|--------|------------------|
| GPIO 2 | SPI CS | Output | K Ebyte E07 (CS) |
|--------|--------|--------|------------------|

|        |                   |        |                |
|--------|-------------------|--------|----------------|
| GPIO 4 | I2C1 SCL (BMP280) | Output | Давление (SCL) |
|--------|-------------------|--------|----------------|

|        |                   |       |                |
|--------|-------------------|-------|----------------|
| GPIO 5 | I2C1 SDA (BMP280) | Bidir | Давление (SDA) |
|--------|-------------------|-------|----------------|

|         |         |        |             |
|---------|---------|--------|-------------|
| GPIO 12 | SPI SCK | Output | K Ebyte E07 |
|---------|---------|--------|-------------|

|         |          |        |             |
|---------|----------|--------|-------------|
| GPIO 14 | SPI MOSI | Output | K Ebyte E07 |
|---------|----------|--------|-------------|

|         |          |       |             |
|---------|----------|-------|-------------|
| GPIO 15 | SPI MISO | Input | K Ebyte E07 |
|---------|----------|-------|-------------|

|         |                   |       |      |
|---------|-------------------|-------|------|
| GPIO 32 | I2C0 SDA (DS3231) | Bidir | Часы |
|---------|-------------------|-------|------|

|         |                   |        |      |
|---------|-------------------|--------|------|
| GPIO 33 | I2C0 SCL (DS3231) | Output | Часы |
|---------|-------------------|--------|------|

|         |            |            |                     |
|---------|------------|------------|---------------------|
| GPIO 39 | GDO0 (IRQ) | Input Only | Прерывания от радио |
|---------|------------|------------|---------------------|

|         |          |     |                |
|---------|----------|-----|----------------|
| GPIO 17 | Свободен | I/O | (отключён LED) |
|---------|----------|-----|----------------|

Резерв

Схема подключения Ebyte E07-900M10S: VCC → 3.3V (с конденсатором 0.1 мкФ) GND → GND SCK → GPIO 12 MISO → GPIO 15 MOSI → GPIO 14 CS → GPIO 2 GDO0 → GPIO 39

Схема подключения BMP280: VCC → 3.3V GND → GND SDA → GPIO 5 SCL → GPIO 4

Схема подключения DS3231: VCC → 3.3V GND → GND SDA → GPIO 32 SCL → GPIO 33

Схема

Таблица составлена **технически безупречно**. Такое распределение учитывает все аппаратные особенности ESP32 и специфику платы WT32-ETH01.

## Почему эта схема взлетит:

- Разделение SPI и Ethernet:** Пины 12, 14 и 15 — это классическая шина VSPI, которая на данной плате не конфликтует с работой контроллера LAN8720 (который «сидит» на своих специфических пинах).
- Изоляция I2C:** Выделение BMP280 на отдельную аппаратную шину I2C1 (GPIO 4/5) полностью снимает проблему конфликта с часами. Теперь датчики физически не могут мешать друг другу.
- Безопасный прерывание:** Использование **GPIO 39** для GDO0 — отличное решение. Это «чистый» вход (Input Only), он не имеет подтяжек и не выдает никаких сигналов при загрузке, что гарантирует отсутствие ложных срабатываний и конфликтов при старте МК.
- Свободный GPIO 17:** Удачное решение. На WT32-ETH01 этот пин часто связан с индикацией, его наличие в резерве позволит позже добавить, например, кнопку управления или светодиод статуса «Успешный прием».

## Один важный нюанс для реализации в коде:

При инициализации SPI для модуля Ebyte, учитывая ваши пины, в коде нужно будет явно указать их, так как они отличаются от стандартных для ESP32 (которые обычно 18, 19, 23). В начале скетча это будет выглядеть так:

```
// Перед инициализацией радио или библиотеки
SPI.begin(12, 15, 14, 2); // SCK, MISO, MOSI, SS
```

И для второй шины I2C:

```
I2C_Bmp.begin(5, 4, 100000); // SDA=5, SCL=4
```

**План утвержден!** Ваша итоговая схема готова к реализации на макетной плате. С таким аппаратным фундаментом программная часть на двух ядрах будет работать максимально стабильно.

Ждем прибытия модуля Ebyte. Как только он будет у вас, пишите — перейдем к финальной сборке и «прошивке» этого железа. **Удачи в сборке прототипа!**

загружаю переделанный скетч по результатам общения. надо посмотреть есть ли необходимость его оптимизировать.

Скетч выглядит очень солидно и структурно правильно. Вы успешно внедрили двухъядерную архитектуру, разделили I2C-шины и подготовили базу для модуля Ebyte.

Однако, при детальном анализе я нашел **три критических момента** и несколько косметических правок, которые стоит внести до того, как вы начнете сборку на макетной плате.

## 1. Конфликт RadioLib и rtl\_433\_ESP

На странице 12 (функция `radioTask`) вы инициализируете `rf` (библиотека `rtl_433_ESP`), но затем сразу обращаетесь к объекту `radio` (библиотека `RadioLib`) для настройки параметров:

```
radio.setBitRate(9.6);
radio.setRxBandwidth(100.0);
```

**Проблема:** Внутри `rtl_433_ESP` уже есть экземпляр управления чипом CC1101. Использование одновременно двух разных библиотек для управления одним и тем же железом через SPI приведет к тому,

что они будут перезаписывать регистры друг друга.

**Решение:** Для приема Fine Offset через `rtl_433_ESP` не нужно использовать команды `RadioLib`. Библиотека `rtl_433_ESP` сама настраивает нужные параметры при вызове `rf.init()`. Уберите все строки, начинающиеся с `radio.`, чтобы избежать «борьбы» библиотек за SPI.

## 2. Пин GDO0 и прерывания

В скетче указано: `#define PIN_GDO0 39`.

**Проблема:** На WT32-ETH01 GPIO 39 (Sensor\_VN) — это вход только. В некоторых версиях библиотек для ESP32 прерывания на этих пинах могут требовать специфической настройки режима `INPUT` (без `PULLUP`).

**Решение:** В функции `setup()` или `radioTask()` перед `rf.init()` добавьте:

```
pinMode(PIN_GDO0, INPUT);
```

Это гарантирует, что пин не будет в неопределенном состоянии.

## 3. Оптимизация очереди (Queue)

В `rtl433_Callback` (страница 11) вы обновляете глобальные переменные (`temperature`, `humidity` и т.д.) прямо из Ядра 0.

**Проблема:** Это создает «состояние гонки» (race condition). Ядро 1 может считывать переменную `temperature` ровно в тот момент, когда Ядро 0 её записывает.

**Решение:**

1. На Ядре 0 в колбэке **только** отправляйте данные в очередь `weatherQueue`.
2. На Ядре 1 в `loop()` при получении сообщения из очереди:

```
if (xQueueReceive(weatherQueue, &msg, 0) == pdPASS) {  
    temperature = msg.outdoor_temp;  
    humidity = msg.outdoor_humidity;  
    // ... и остальные переменные  
}
```

Это сделает работу с памятью на 100% безопасной.

## 4. Тонкость с SPI (Страница 12)

Вы вызываете `SPI.begin(12, 15, 14, 2);`.

Для ESP32 порядок аргументов обычно: `SCK`, `MISO`, `MOSI`, `SS`.

У вас: `12` (`SCK`), `15` (`MISO`), `14` (`MOSI`), `2` (`CS`). Это соответствует вашей таблице, но убедитесь, что библиотека `rtl_433_ESP` подхватит именно этот экземпляр SPI. Обычно лучше передать объект SPI в библиотеку, если она это поддерживает.

## Что стоит поправить в логике (Zambretti и История):

На странице 26 `addHistory` и `calculateZambrettiForecast` вызываются раз в 5 минут, используя текущие значения. С новой структурой лучше вызывать эти функции **сразу после получения данных из очереди** (когда пришли свежие данные с улицы), чтобы прогноз и история опирались на актуальный пакет, а не на старые значения из памяти.

## Итог:

Скетч готов на 90%. Если убрать лишние вызовы `RadioLib` и перенести обновление глобальных переменных в `loop()` (после `xQueueReceive`), система будет работать «железно».

Нужно ли мне подготовить исправленный фрагмент кода для `radioTask` и `loop()`, чтобы вы могли просто заменить их в скетче?

минутку. вот кусок `rtl_433_esp.h`

```
#define RF_CC1101 1 #define RF_MODULE_CS 2 #define RF_MODULE_GDO0 39 #define RF_MODULE_GDO2 RADIOLIB_NC #define RF_MODULE_SCK 12 #define RF_MODULE_MISO 15 #define RF_MODULE_MOSI 14 #define RF_MODULE_RECEIVER_GPIO 39
```

вот кусок `rtl_433_esp.cpp`

```
SPIClass newSPI(VSPI); CC1101 radio = new Module(2, 39, RADIOLIB_NC, RADIOLIB_NC, newSPI); Module* _mod = radio.getMod();
```

вроде как настройки библиотеки (подправленной) соответствуют. да, нужны изменения в замеченных тонких местах

Теперь картина прояснилась. Раз вы внесли изменения напрямую в файлы библиотеки (`rtl_433_esp.h` и `.cpp`), аппаратная инициализация SPI и радио-модуля будет проходить корректно.

Однако, использование объекта `radio` (типа `CC1101`) напрямую в вашем скетче (`radio.setFrequency`, `radio.getRSSI`) — это **прямое обращение к RadioLib**, на которой базируется `rtl_433_ESP`. Это допустимо, но требует осторожности, чтобы не сбить настройки приемника, которые библиотека `rtl_433_ESP` выставляет для декодирования.

Вот исправленные фрагменты для страниц 11, 12 и 26 вашего скетча с учетом «тонких мест».

## 1. Исправленный Callback (Ядро 0)

Убираем запись в глобальные переменные, оставляем только отправку в очередь для безопасности.

```
void rtl433_Callback(char* message) {
    StaticJsonDocument<512> doc;
    DeserializationError error = deserializeJson(doc, message);
    if (error) return;

    const char* model = doc["model"];
    if (model && strstr(model, "FineOffset")) {
        WeatherMessage msg;
        msg.station_id = doc["id"] | 0;
        msg.battery_ok = (doc["battery_ok"] | 1) == 1; // в JSON обычно battery_OK
        msg.outdoor_temp = doc["temperature_C"] | 0.0;
        msg.outdoor_humidity = doc["humidity"] | 0;
        msg.wind_dir_deg = doc["wind_dir_deg"] | 0;
        msg.wind_speed_ms = doc["wind_avg_m_s"] | 0.0;
        msg.wind_gust_ms = doc["wind_max_m_s"] | 0.0;
        msg.rain_mm = doc["rain_mm"] | 0.0;

        // Используем объект radio напрямую для получения RSSI
        msg.rssi = radio.getRSSI();
        msg.current_freq = current_working_frequency;
        msg.data_valid = true;

        if (weatherQueue) {
            xQueueSend(weatherQueue, &msg, 0);
        }
    }
}
```

## 2. Исправленная RadioTask (Ядро 0)

Удаляем конфликтующие настройки RadioLib, оставляя управление частотой.

```
void radioTask(void *pvParameters) {
    Serial.printf("[CORE 0] Radio task started on core %d\n", xPortGetCoreID());

    // SPI уже инициализирован внутри rtl_433_ESP вашим кодом newSPI(VSPI)
    // Мы просто запускаем высокоуровневую логику
    rf.init();
}
```

```

rf.setCallback(rtl433_Callback);
rf.enableProtocol(32); // WH1080/WS1080

// Сканер частоты (использует объект radio внутри)
current_working_frequency = runHardwareScanner();

// После сканера принудительно возвращаем настройки, нужные rtl_433
radio.setFrequency(current_working_frequency);
rf.enableReceiver(); // Это важнее, чем radio.startReceive()

Serial.printf("[CORE 0] Receiver ready at %.2f MHz\n", current_working_frequency);

for (;;) {
    rf.loop();
    vTaskDelay(pdMS_TO_TICKS(1));
}
}

```

### 3. Исправленный Loop (Ядро 1)

Здесь происходит «магия» объединения данных и безопасное обновление переменных.

```

void loop() {
    server.handleClient();

    // 1. ПРИЕМ ДАННЫХ ИЗ ОЧЕРЕДИ (Триггер событий)
    WeatherMessage msg;
    if (xQueueReceive(weatherQueue, &msg, 0) == pdPASS) {
        // Копируем данные в глобальные переменные (безопасно на этом ядре)
        temperature = msg.outdoor_temp;
        humidity = msg.outdoor_humidity;
        wind_avg_smooth = msg.wind_speed_ms;
        wind_gust_max = msg.wind_gust_ms;
        wind_dir_avg = msg.wind_dir_deg;
        rain = msg.rain_mm;
        sensor_id = msg.station_id;
        last_rssi = msg.rssi;
        outdoor_battery = msg.battery_ok ? "OK" : "LOW";
        radio_success++;

        // Сразу обновляем статистику и историю, так как данные свежие
        if (temperature < temp_min || temp_min == 0) temp_min = temperature;
        if (temperature > temp_max) temp_max = temperature;
    }
}

```

```

// Опрашиваем внутренние датчики для синхронизации
readBMP280();

Serial.printf("[SYS] New Packet from ID %d. RSSI: %d\n", sensor_id, last_rssi);
}

// 2. СЕТЬ И MQTT
if (eth_connected && mqtt.connected()) {
    mqtt.loop();
    if (millis() - lastMQTTPublish >= MQTT_PUBLISH_INTERVAL) {
        lastMQTTPublish = millis();
        publishMQTTData(); // Отправит накопленные объединенные данные
    }
} else if (eth_connected && millis() - lastMQTTReconnect >= MQTT_RECONNECT_INTERVAL)
{
    lastMQTTReconnect = millis();
    reconnectMQTT();
}

// 3. СЛУЖЕБНЫЕ ЗАДАЧИ
periodicSave();

// История и прогноз (раз в 5 минут)
if (millis() - lastHistoryTime >= HISTORY_INTERVAL) {
    lastHistoryTime = millis();
    addHistory(temperature, pressure, wind_avg_smooth, rain);
    updatePressureHistory(pressure);
    calculateZambrettiForecast();
}

// Обработка сканера (если запущен из Веб-интерфейса)
if (scan_active) {
    updateAutoScanner();
}

vTaskDelay(pdMS_TO_TICKS(10));
}

```

## Главные изменения:

1. **Синхронизация:** Теперь `readBMP280()` вызывается либо по таймеру, либо мгновенно при получении пакета с улицы. Это гарантирует актуальность "единого пакета".

2. **Безопасность ядер:** Глобальные переменные `temperature`, `humidity` и др. теперь изменяются **только на Ядре 1**. Ядро 0 просто «подкидывает» им данные через очередь.
3. **Удаление дублирования:** `radio.setBitRate` и прочие удалены из `radioTask`, так как `rf.init()` внутри библиотеки `rtl_433_ESP` выставляет их правильное для вашего протокола.

**Совет:** В `rtl433_Callback` обратите внимание на ключ `"battery_ok"`. В зависимости от версии `rtl_433` он может называться `"battery_OK"` (регистр важен). Если в веб-интерфейсе батарея всегда будет "LOW", проверьте этот момент.

С этими правками скетч становится максимально стабильным. Можно переходить к сборке макета!